Computacíon Paralela y Distribuída

2025-II

José Fiestas

23 de octubre de 2025

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe

Práctica Dirigida 04:

Unidad 4: OMP

PD04: 5 pts (avance presencial)

Tarea: 1 pt (no incluye avance)

Sistema de evaluación 2025-II

Evaluación Contínua (TA_1, TA_2):

Se desarrolla en **grupos de 2 o 3**, en clase de Laboratorio. Está permitido cambiar de grupo durante el semestre aunque se recomienda mantenerlo. Cada evaluación consiste de:

- una **Practica Dirigida (PD)**, que se entrega el mismo día de la evaluación, al final de la clase. Debe desarrollar las preguntas planteadas y completar el 80 % de la nota
- una **Tarea**, que se entrega unos días después de la PD, y debe completar el desarrollo de los ejercicios planteados, por el 20 % restante de la nota

En el **Anexo Detallado de Evaluaciones** en Canvas se indica el puntaje para cada forma de evaluación y los temas de la PD

Ejercicio 1: Gradiente Logística en paralelo (2.5 pts)

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^n$ con $x_i \in \mathbb{R}^d$ y $y_i \in \{0, 1\}$, la regresión logística busca un vector de pesos $w \in \mathbb{R}^d$ que modele la probabilidad condicional de que $y_i = 1$ dado x_i como:

$$p_i = P(y_i = 1 \mid x_i; w) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \quad \text{donde } z_i = w^\top x_i.$$

La función de pérdida promedio (log-loss o entropía cruzada) es:

$$L(w) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

El gradiente de esta función con respecto a los pesos es:

$$\nabla_w L(w) = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i) x_i.$$

Por lo tanto, el paso de actualización mediante descenso de gradiente se define como:

$$w \leftarrow w - \eta \nabla_w L(w),$$

donde $\eta > 0$ es la tasa de aprendizaje.

El algoritmo recorre todas las muestras, acumula el gradiente $\nabla_w L(w)$ y actualiza el vector w una vez al final del ciclo.

Ejercicio 1: Gradiente Logística en paralelo (cont.)

Algorithm 1: Gradiente_Logística(X, y, η, n, d)

Input: $X \in \mathbb{R}^{n \times d}$: matriz de datos, $y \in \{0,1\}^n$: etiquetas binarias
Output: Vector de pesos $w \in \mathbb{R}^d$ actualizado
Inicializar $w_j \leftarrow 0$, $grad_j \leftarrow 0$, para $j = 1, \ldots, d$; $loss \leftarrow 0$;
for $i \leftarrow 1$ **to** n **do**
    $z \leftarrow 0$;
    for $j \leftarrow 1$ **to** d **do**
        $z \leftarrow z + w_j \cdot X_{i,j}$;
    end
    $p \leftarrow \dfrac{1}{1 + e^{-z}}$; $err \leftarrow p - y_i$;
    $loss \leftarrow loss - \big(y_i \log(p + \varepsilon) + (1 - y_i) \log(1 - p + \varepsilon)\big)$;
    for $j \leftarrow 1$ **to** d **do**
        $grad_j \leftarrow grad_j + err \cdot X_{i,j}$;
    end
end
for $j \leftarrow 1$ **to** d **do**
    $w_j \leftarrow w_j - \dfrac{\eta}{n} \cdot grad_j$;
end
return w, $loss/n$;

Ejercicio 1: Gradiente Logística en paralelo (cont.)

X es una matriz de tamaño $n \times d$. **n** es la cantidad de muestras, y **d** las características. Cada fila $x_i \in \mathbb{R}^d$ representa una muestra o ejemplo con d características. **Y** es un vector de tamaño n. $y \in \{0, 1\}$ (regresión logística binaria). η es la tasa de aprendizaje (learning rate).

Paralelice el código adjunto (*ejercicio01_sec.cpp*) con OMP. Para ello, decida primero que bucle paralelizar, dependiendo de su complejidad. Recuerde que *n>>d*. Genere los siguientes códigos:

1.1) usando la clausula reduction **(1 pt)**

1.2) usando un constructor critical **(1 pt)**

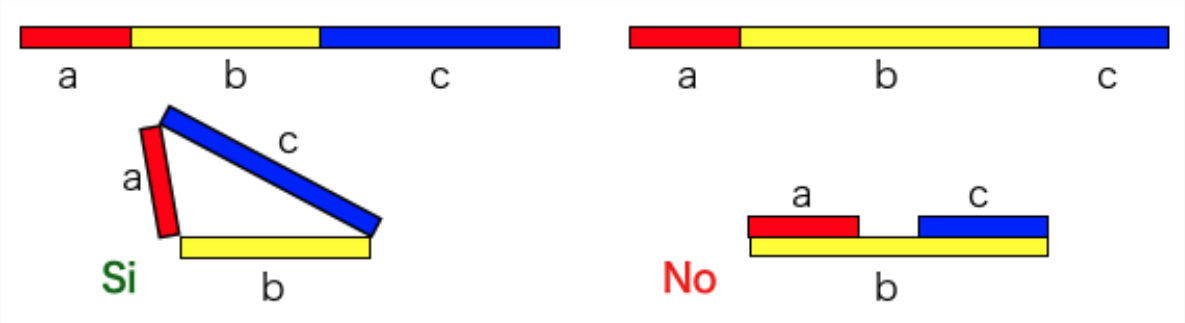
1.3) usando un constructor atomic **(0.5 pts)**

Compare el performance entre los códigos midiendo tiempos de ejecución en cada caso. Asegure la validez de los códigos al confirmar el resultado obtenido en su versión secuencial.

Ejercicio 2: Broken triangle (2 pts)

Resuelva el problema **broken triangle** utilizando un modelo Monte Carlo en paralelo. El problema consiste en lo siguiente:
Dos puntos aleatorios en un intervalo dividen al mismo en tres segmentos
a) ¿Cúal es la probabilidad de que los 3 segmentos formen un triángulo?
b) ¿Cúal es la probabilidad de que este triángulo sea obtuso?
Si los lados son s_1,s_2,s_3, un triángulo debe satisfacer las condiciones $s_1 \leq s_2 + s_3$, $s_2 \leq s_1 + s_3$, $s_3 \leq s_1 + s_2$, o (en código):

$$max(s_1, s_2, s_3) < 0.5$$



Ejercicio 2 (cont.)

Un triángulo es obtuso, si uno de sus ángulos internos es más de 90°. Es decir, si $s_{max}^2 \geq s_i^2 + s_j^2$, o (en código):

$$2 \cdot s_{max}^2 > s_1^2 + s_2^2 +, s_3^2$$

Siendo s es lado de mayor longitud.

1. Paralelice el código adjunto (*ejercicio02_sec.cpp*) usando el paradigma de memoria compartida (OMP). Ejecute el secuencial y responda a las preguntas a) y b). Compruebe que los resultados en paralelo coinciden con el secuencial **(1 pt)**
2. Describa las ventajas de la implementación usada según lo discutido en clase. Describa en qué consiste el overhead en el código propuesto ¿Puede el algoritmo generado aún ser optimizado? **(1 pt)**

Ejercicio 3 (DAG-OMP) (1.5 pts)

En la PD2 se generó el DAG del siguiente algoritmo

```
double sistema() {
double A[n][n],B[n][n],a,b,x,y,z;
ini(A,B);
a = tarea_1(A);
b = tarea_2(B);
s1 = suma_prefijos1(B,a);
s2 = suma_prefijos2(B,a,b);
 z = s1 + s2;
 return z;
}
```

La función ini() inicializa dos matrices. Los parámetros del resto de funciones son sólo de entrada (no se modifican). Genere un código C++ -OMP que paralelice eficientemente este algoritmo, utilizando **tasks** con cláusulas **depend**. Complete las funciones adecuadamente **(1 pt.)**. Genere el DAG correspondiente **(0.5 pts.)**